

FIAP · MBA AI ENGINEERING & MULTI-AGENTS
CLOUD & COGNITIVE ENVIRONMENTS

Storage & Bancos de Dados na Nuvem

Aula 02 — Onde os dados dos agentes vivem: object storage,
SQL relacional, NoSQL e a primeira busca vetorial

Como vamos passar as próximas 4 horas

00:00 — 01:00

Bloco 1 — Storage

Object / File / Block, Blob tiers, redundância e lifecycle + lab Terraform

01:00 — 02:00

Bloco 2 — Relacional & Key Vault

Azure SQL Free Offer, connection pooling, segredos + lab Python

02:00 — 02:10

Break

10 minutos de pausa

02:10 — 03:35

Bloco 3 — NoSQL & busca vetorial

Cosmos DB, Vector DBs e RAG + lab Cosmos & Azure AI Search

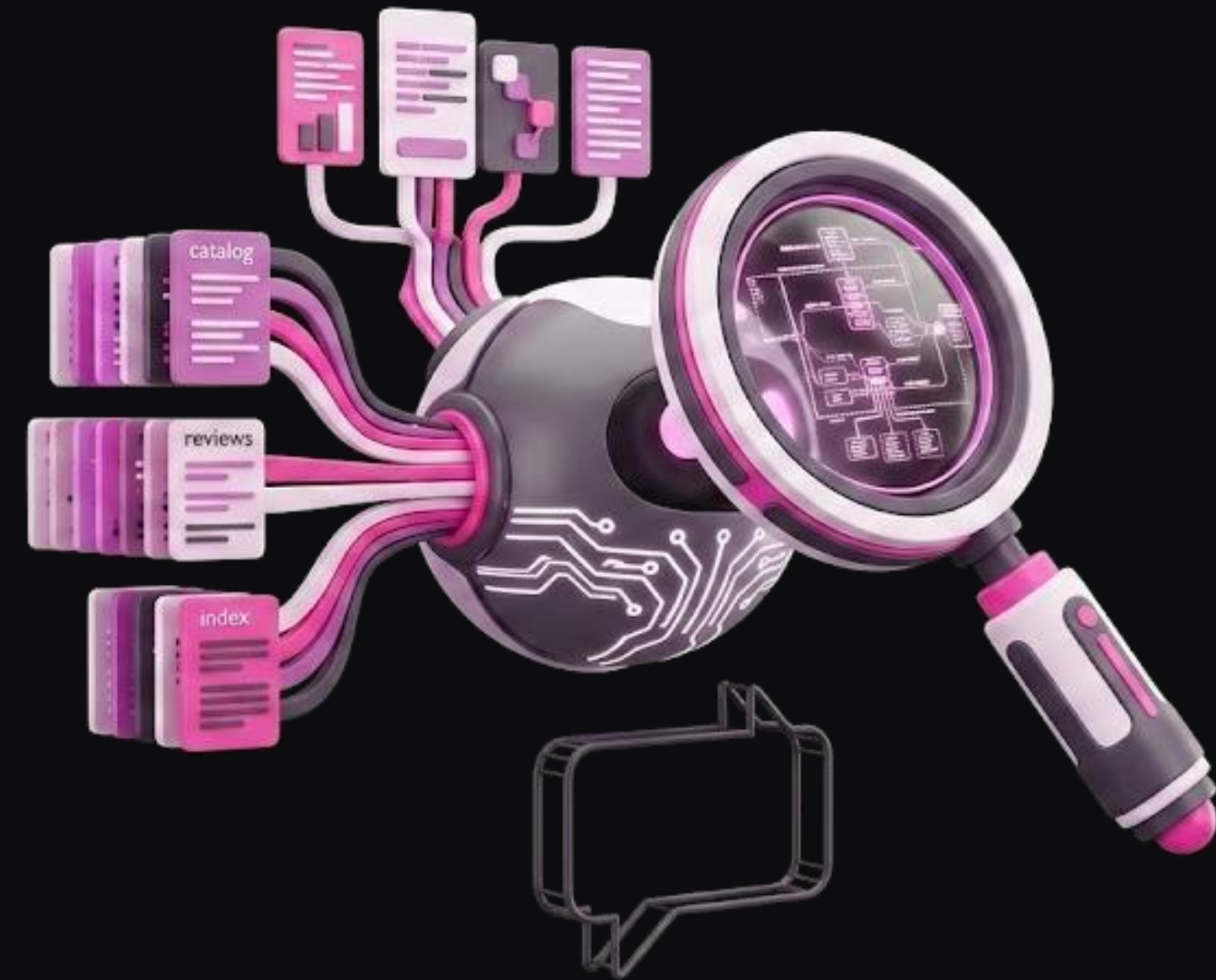
03:35 — 04:00

Bloco 4 — Analytics & wrap-up

OLTP vs OLAP, Synapse/Fabric, destroy e a arquitetura de dados da QC

Um agente sem dados é só um **chatbot genérico.**

Na Aula 1 montamos a casa vazia — Resource Group, VM, IaC. Hoje a gente mobília: decide onde vão viver o catálogo, as reviews e o índice que faz a busca semântica de uma empresa funcionar.



Onde vão viver os dados dos agentes ?

Storage e bancos são decisões arquiteturais que definem o que o agente consegue ou não fazer.

01



Armazenamento na nuvem

Os 3 tipos de storage, os tiers do Blob, redundância, lifecycle — e o primeiro provisionamento via Terraform.

Antes do banco, onde o byte mora

OBJECT STORAGE

Blob, S3, GCS

Dados não-estruturados — imagens, vídeos, logs, backups.

Acesso via HTTP/REST. Escala para petabytes.

imagens, vídeos, logs, backup

FILE STORAGE

Azure Files, Elastic File System, File Store

Drives compartilhados montados como **/mnt/dados** em várias VMs. Substitui o servidor de arquivos tradicional.

share entre VMs

BLOCK STORAGE

Managed Disks, Elastic Block Storage, Persistent Disk

Disco virtual atrelado a uma VM — SO, banco, qualquer coisa que exija **baixa latência**.

Não compartilhável ao mesmo tempo.

disco de uma VM

· PARA REFLETIR ·

Todo dado mora em algum **lugar** **físico.**

A nuvem não é etérea — são discos girando num galpão refrigerado.
Escolher o tipo de storage é escolher como esse galpão te cobra.

A nuvem não é etérea —

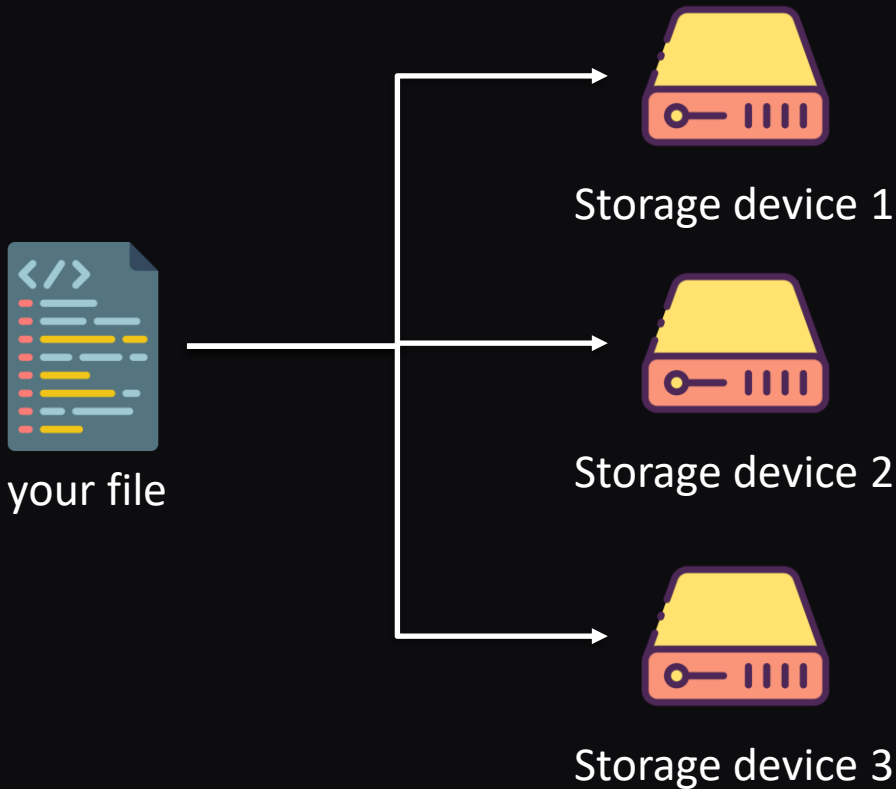
Blob Storage é um sistema de armazenamento de objetos distribuído, projetado para armazenar e recuperar grandes volumes de dados não estruturados de forma altamente escalável. Diferentemente de um sistema de arquivos tradicional, os dados não são organizados em blocos ou hierarquias complexas de diretórios, mas sim como objetos independentes identificados por uma chave única e acessados via APIs HTTP.

Diferente de arquivos armazenados no teu HD, armazenamento de objetos **NÃO** são armazenados em pastas/subpastas com arquivos.

BLOB EM PROFUNDIDADE

Chave (key)	Dados (Payload)	metadados
path1	01101101 ...	{size, created, etc}
path2	10110011 ...	{size, created, etc}
path3	11011101 ...	{size, created, etc}

Para garantir durabilidade, e velocidade os objetos são copiados automaticamente.



Em arquiteturas modernas de dados, o Blob Storage é o "disco rígido da nuvem". Ele não executa processamento; sua função é armazenar dados com alta durabilidade e escalabilidade para que motores como Spark, Databricks, Synapse, Snowflake ou ferramentas de IA possam consumi-los sob demanda.

Durabilidade típica:
11 noves (99,999999999%)
16 noves em alguns provedores

Chave (key)	Dados (Payload)	metadados
path1	01101101 ...	{size, created, etc}
path2	10110011 ...	{size, created, etc}
path3	11011101 ...	{size, created, etc}

/pastaraizA/subpastaA1/subpastanivel2/arquivo1.csv
/pastaraizA/subpastaA2/subpastanivel2/arquivo2.csv
/pastaraizA/subpastaA2/subpastanivel2/arquivo1.csv

Em BLOB, NÃO existem pastas e sub-pastas, ao invés disso, existe o conceito de chave e prefixo;

As referências a pastas são apenas para fins didáticos.

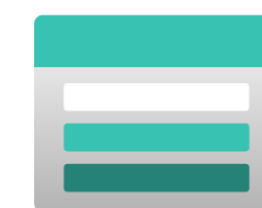
Regras de armazenamento, e processamentos “em lote”, são feitos de acordo com prefixo.



Quando se trabalha com blob profissionalmente, adota-se a prática de nunca armazenar arquivos diretamente na “conta de armazenamento”;

Este fundamento é tão forte que a Azure (diferente de GCP e AWS), adotam o conceito de “container” de armazenamento, que nada mais é do que a pasta raiz da conta de armazenamento;

Isso impede que arquivos fiquem soltos fora de uma pasta raiz.



Storage Account



Storage Container

Storage Container

Storage Container

O mesmo byte, quatro preços (exemplo)

TIER	CUSTO /GB/MÊS	QUANDO USAR
Hot	~ \$0,018	Acesso frequente — APIs, dados quentes, catálogo servido aos agentes
Cool	~ \$0,010	Backup recente, dados de 30+ dias sem acesso regular
Cold	~ \$0,0036	Backup antigo, compliance — mínimo de 90 dias de retenção
Archive	~ \$0,002	Cold storage real — leva minutos para recuperar os arquivos

Hot → Archive é 9× mais barato pelo mesmo byte. A decisão não é técnica — é financeira.

*Custos de referência dezembro de 2025

** Classes de referência podem mudar de acordo com o provedor

O dado esfria sozinho

0–30 DIAS

Hot

logs de compra recém-criados, detecção de fraude

30–90 DIAS

Cool

consultas esporádicas, relatórios mensais

90+ DIAS

Archive

compliance LGPD, retenção fiscal de anos

Uma política declarativa (policy) move blobs por tier **sem ninguém tocar**.

Em escala de TB, vira **seis dígitos por ano**.

ITAÚ · MAGALU

Data lakes em camadas raw / trusted / refined sobre Blob — base de todo o analytics.

NETFLIX

CDN + Object Storage: Hot para títulos populares, Cool para o catálogo antigo.

PADRÃO DE MERCADO

Imagens de produto em Hot, logs de compra Cool → Archive depois de 90 dias.

► ORGANIZAÇÃO DE DATACENTERS - DC

quantas cópias, onde



O datacenter é a unidade física concreta: o edifício que abriga os servidores (compute), os sistemas de armazenamento (storage), os equipamentos de rede, além de toda a infraestrutura de suporte — geradores, no-breaks, climatização, segurança física.

Datacenters próximos (de 0,5 a 5 km) são considerados como se estivessem na mesma zona e possuem link de baixíssima latência entre si.

Quando estão organizados em distâncias maiores (~50+ km), também possuem link de conexão, mas a própria distância se encarrega de adicionar latência adicional.

Uma região é um agrupamento de zonas ou DCs dentro de uma grande região física ou país



Quantas cópias do seu dado existem?

LRS

**3 cópias,
1 datacenter**

Locally Redundant.

Barato. Sem proteção
contra desastre
regional.

ZRS

3 zonas da região

Zone Redundant.

Sobrevive à queda de
uma zona inteira.

GRS

**LRS + região
secundária**

Geo Redundant.

Proteção contra
desastre regional
completo.

RA-GRS

GRS com leitura

Lê da região secundária
mesmo sem failover.

Menor resiliência.

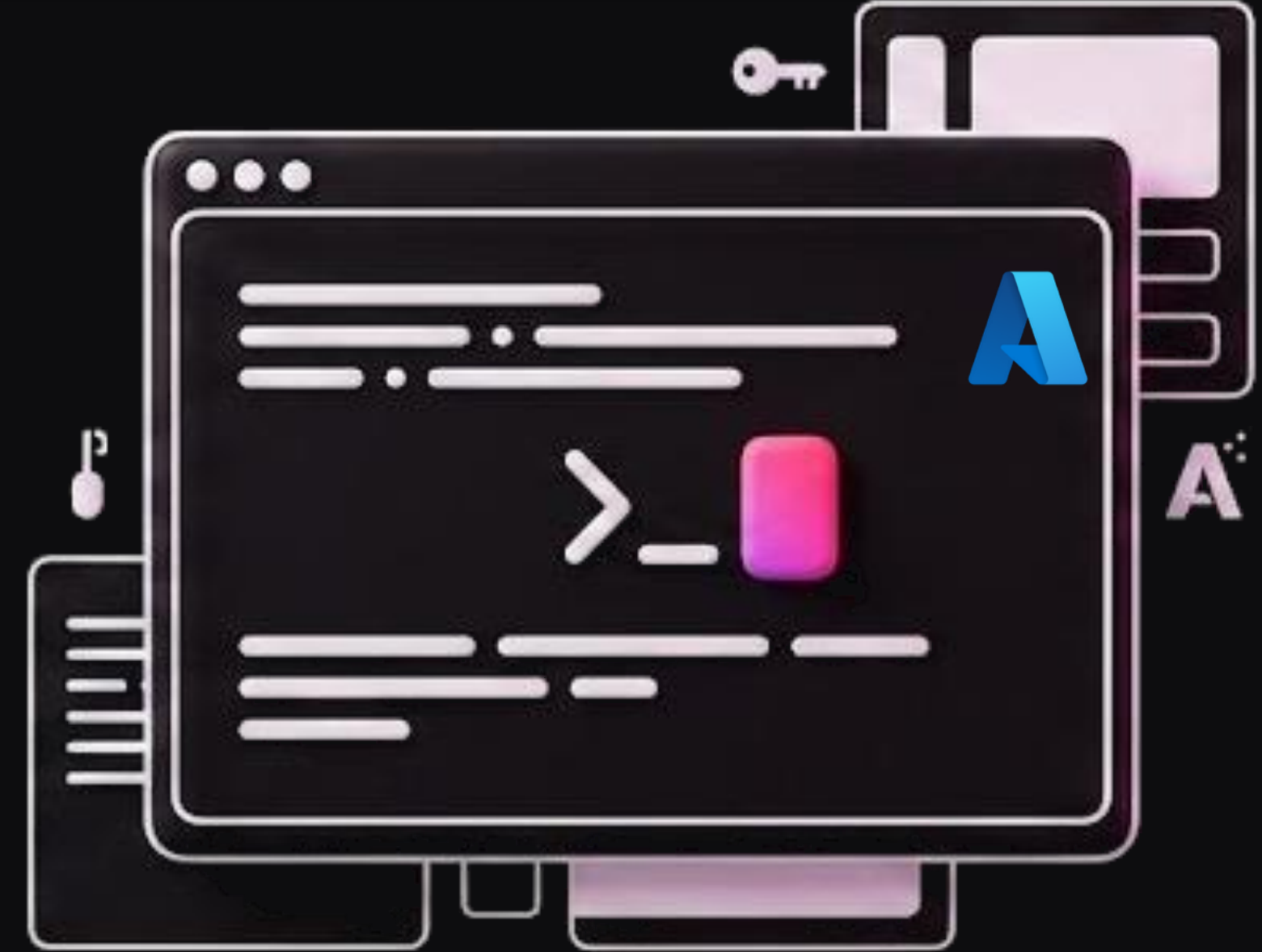
► BLOCO 1 · Petabytes não impressionam mais

Big big big data



Talvez exabytes continuem impressionando por mais uns 5 anos.

LAB 1



Storage + Container (de armazenamento)

Antes de continuar, certifique-se de ter à disposição:

- Conta Azure já logada
- Cloud Shell funcionando
- Uma boa xícara de café ☕

1 – Preparação – cria toda a camada de dados

Clonar o repositório (apenas na primeira vez)

```
git clone https://github.com/elthonf/aie-cloud.git  
cd aie-cloud/aulas/02-storage-bancos/lab/terraform
```

Abrir o editor VS Code na pasta do lab
code .

Gerar senha forte para o admin do SQL (não use senha trivial)

```
SQL_PASSWORD=$(openssl rand -base64 24)  
echo "Senha gerada (guarde em local seguro): $SQL_PASSWORD"
```

Inicializar providers (download de bibliotecas)

```
terraform init
```

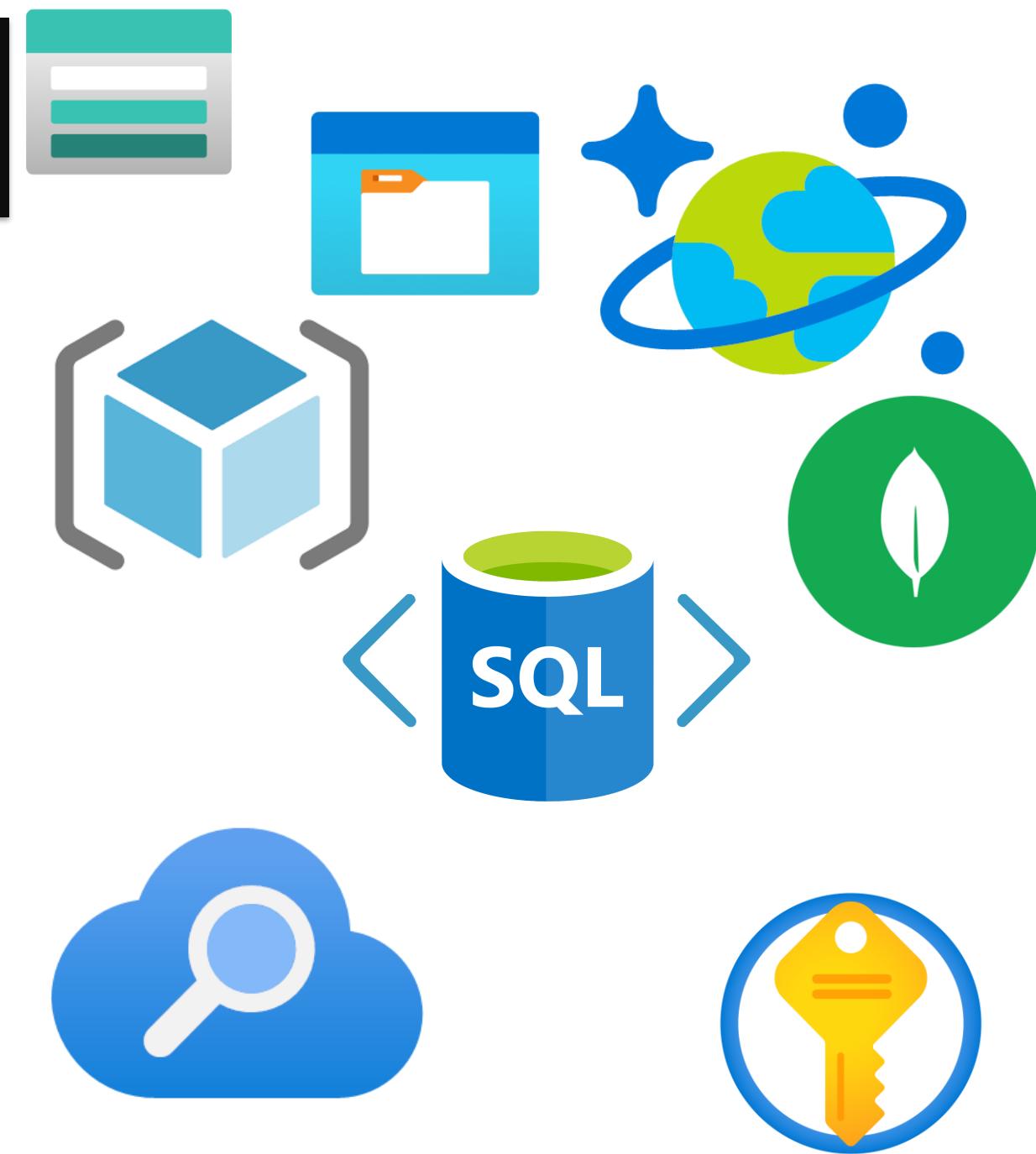
Ver o que será criado

```
terraform plan -var="sql_admin_password=$SQL_PASSWORD"
```

Aplicar (digite 'yes' se perguntar)

```
terraform apply -auto-approve -var="sql_admin_password=$SQL_PASSWORD"
```

Caso o repo tenha sido clonado, apague antes com
`rm -rf aie-cloud`



Aproximadamente ~5 a ~8 minutos (tempo para explorar a pasta terraform)

Lembrete da regra de ouro:

Ao final, devemos encerrar tudo com **Exclusão do RG** ou com `terraform destroy`

Se necessário, bash que pode te apoiar em buscar regiões:

```
check_regions_cosmos.sh  
check_regions_search.sh  
check_regions_sqlserver.sh
```

2 – Enquanto o terraform trabalha, vamos explorar o que temos nos arquivos “.tf” da aula 2

aie-cloud/aulas/02-storage-bancos/lab/terraform

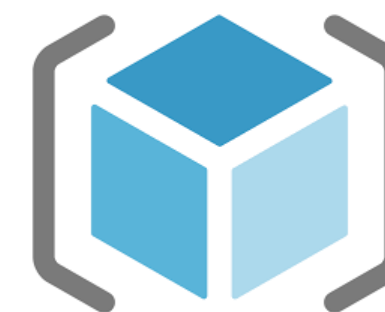
Arquivo	O que define
main.tf	Providers, sufixo aleatório, Resource Group, locals
variables.tf	location e sql_admin_password
outputs.tf	Nomes e endpoints consumidos pelos scripts Python
storage.tf	Storage Account + 3 containers (catalogo, imagens, logs) + lifecycle
sql.tf	SQL Server + Database (GP Serverless, auto-pause) + firewall rules
keyvault.tf	Key Vault + RBAC + segredo da connection string
cosmos.tf	Cosmos DB Account (Serverless) + DB + container
search.tf	AI Search service (SKU Free) + 2 role assignments

Armazena em variáveis valores criados pelo terraform

```
export STORAGE_ACCOUNT_NAME=$(terraform output -raw storage_account_name)
export KEY_VAULT_NAME=$(terraform output -raw key_vault_name)
export COSMOS_ENDPOINT=$(terraform output -raw cosmos_endpoint)
export SEARCH_ENDPOINT=$(terraform output -raw search_endpoint)
export MONGO_IP=$(terraform output -raw mongodb_public_ip)
```

Exibe em tela, variáveis criadas pelo terraform

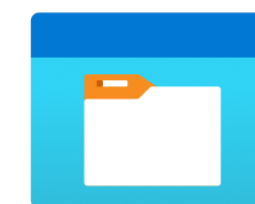
```
echo "Storage: $STORAGE_ACCOUNT_NAME"
echo "Key Vault: $KEY_VAULT_NAME"
echo "Cosmos: $COSMOS_ENDPOINT"
echo "Search: $SEARCH_ENDPOINT"
echo "MongoDB: $MONGO_IP:27017 | Mongo Express: $(terraform output -raw mongo_express_url)"
```



Resource Group



Storage Account



Storage Container



SQL Server + Database



Key Vault



Cosmos DB Account



AI Search

► LAB 1 – Upload do arquivo “produto.csv” para a conta de armazenamento

- 1 – Localize o arquivo data/produtos.csv usando o VS Code;
- 2 – Explore o arquivo e veja seu conteúdo;
- 3 – Conceda acesso de “Storage Blob Data Contributor” para seu login;
- 4 – Faça o upload para o container “catalogo” dentro da conta de armazenamento;
- 5 – Confirme se o arquivo está no container.

Cria role de “Contributor” para seu login (aguarde 30s)

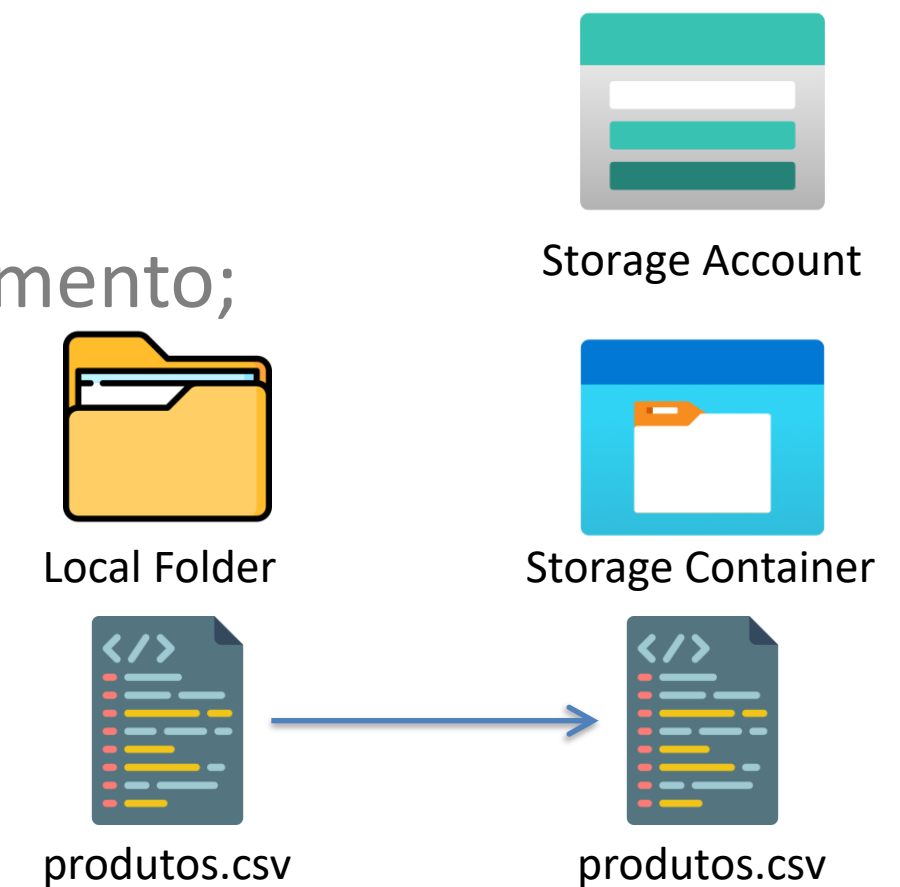
```
az role assignment create \  
  --assignee $(az ad signed-in-user show --query id -o tsv) \  
  --role "Storage Blob Data Contributor" \  
  --scope $(az storage account show -n "$STORAGE_ACCOUNT_NAME" --query id -o tsv)
```

Após localizar o arquivo, faça o upload com:

```
az storage blob upload \  
  --account-name "$STORAGE_ACCOUNT_NAME" \  
  --container-name catalogo \  
  --name produtos.csv \  
  --file ~/aie-cloud/aulas/02-storage-bancos/lab/data/produtos.csv \  
  --overwrite
```

Listar os arquivos do container:

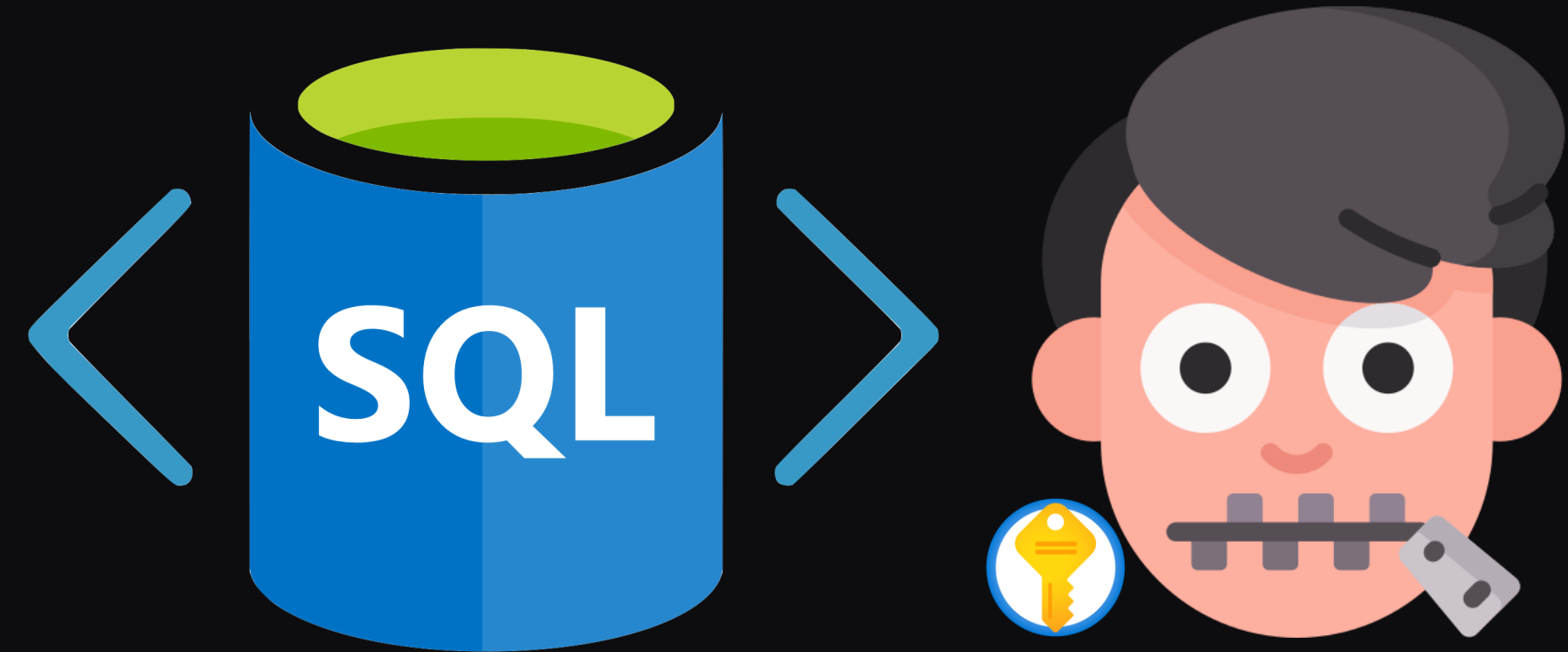
```
az storage blob list \  
  --account-name "$STORAGE_ACCOUNT_NAME" \  
  --container-name catalogo \  
  --output table
```



Dica:

Refaça o laboratório, e acompanhe na **interface gráfica** o que está acontecendo.

02



Relacional e Segredos

Por que ainda usamos SQL, o Azure SQL, e a **vacina** da disciplina: Key Vault + Managed Identity.

Por que o relacional não morre

USE RELACIONAL QUANDO

Transações precisam de garantia

ACID — atomicidade, consistência, isolamento, durabilidade. Faturamento, estoque, pedido.

Ou seja, importa onde dinheiro está em jogo (carrinho, pedido, pagamento):

Transações

SQL é linguagem universal — todo time entende.

Modelagem por entidades + relacionamentos: **JOINS**

● catálogo & pedidos

EVITE QUANDO

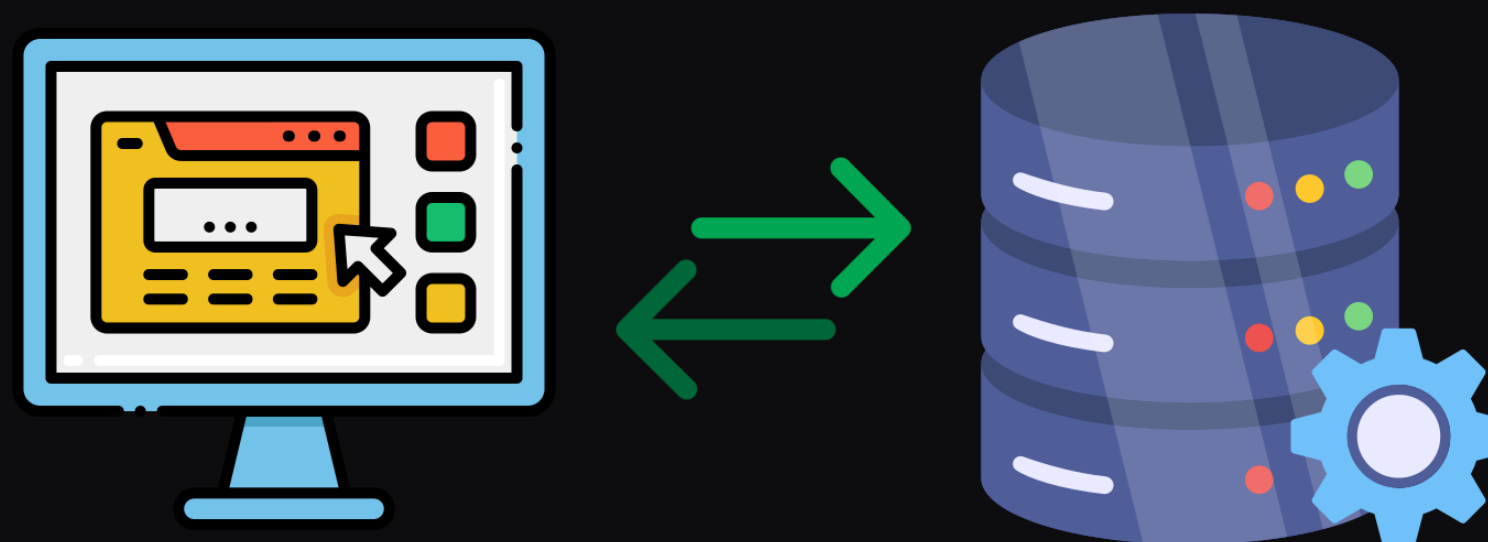
O esquema não para de mudar

Dados massivamente desestruturados — texto livre, JSON variável.

Escala global com latência <10ms, ou dados 100% time-series.

→ aí entra o NoSQL

Uma **conexão SQL** é um **canal de comunicação estabelecido** entre uma aplicação e um banco de dados relacional.



Aplicação

- Abertura de conexão TCP
- Handshake de rede
- Autenticação
- Negociação de segurança (TLS)
- Criação de sessão no banco

Esse processo pode levar dezenas ou centenas de milissegundos. Imagine uma API recebendo 1.000 requisições por minuto:

Requisição 1 → Abre conexão → Executa SQL → Fecha conexão

Requisição 2 → Abre conexão → Executa SQL → Fecha conexão

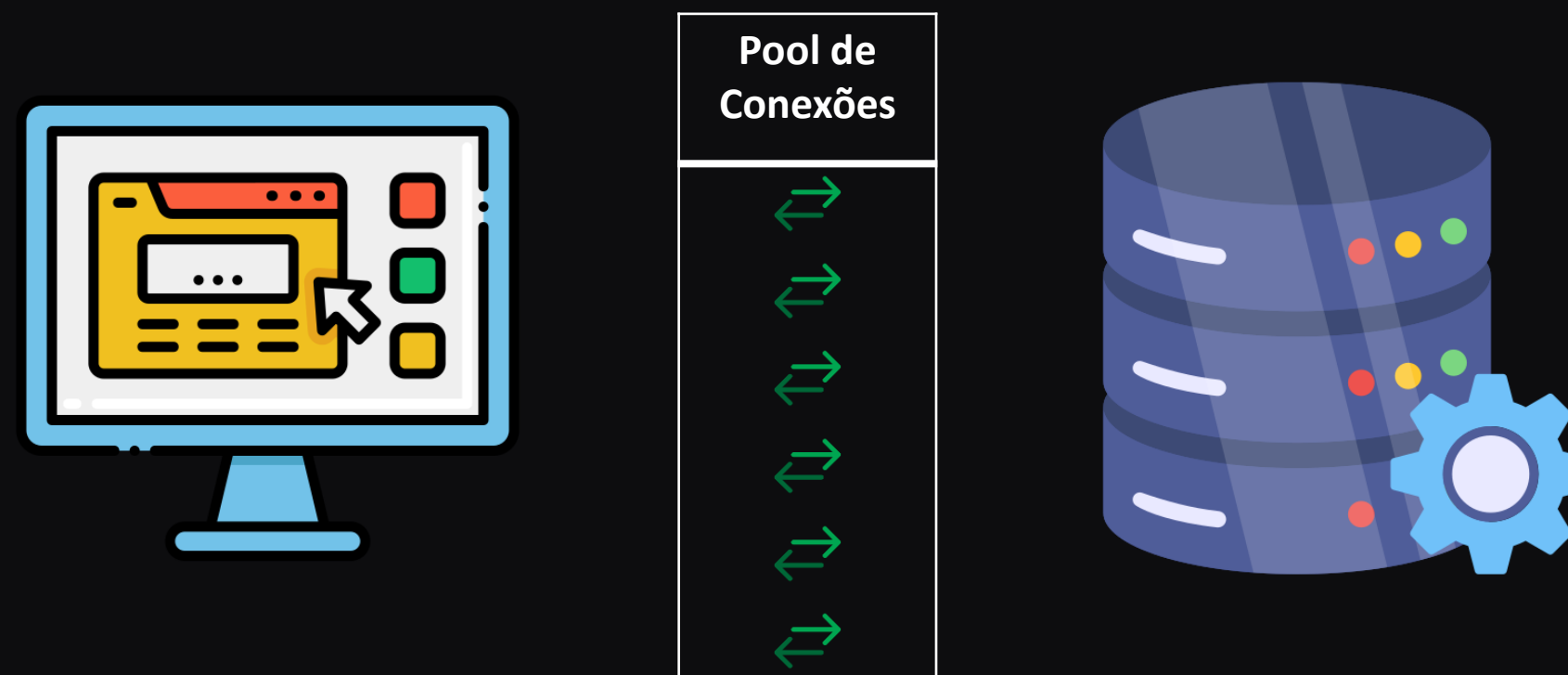
Requisição 3 → Abre conexão → Executa SQL → Fecha conexão

...



Um handshaking é pouco,
mas milhares tomam tempo

Em soluções distribuídas de nuvem como as feitas por agentes, Pools de Conexões são essenciais, não se trata de algo opcional.



Em um pool, as conexões permanecem abertas e são reutilizadas por múltiplas requisições.

Cada conexão tem overhead (TCP + auth)

Pool mantém N conexões prontas, reaproveitadas

Em apps web e Functions (como mcp*), pool é essencial

- Sem pool: 1k req = 1k conexões = banco morre
- Com pool: 1k req = ~20 conexões reusadas

O pool pode ser interno, por aplicação de instância rodando, ou externo, em um servidor de conexões

*mais a frente
você verão mcp



Na imagem acima, qual situação representa o uso de pool de conexões em um banco relacional?

· VACINA ·

A senha no código vaza no **primeiro** **push.**

Já vi engenheiro sênior hardcodear connection string num PR.

Esse é o erro mais comum — e o mais caro — da nuvem pública.



A senha (o segredo) nunca toca no código

✗ ANTES

Connection string hardcoded

Vaza no Git, vaza no log, vaza no incident. Qualquer um com acesso ao repo tem o banco.

```
conn = pyodbc.connect(  
    "Server=...;"  
    "User=admin;"  
    "Password=secret123;"  
)
```

✓ DEPOIS

Key Vault + Managed Identity

O Vault guarda o segredo. A identidade gerenciada do recurso lê — sem nunca conhecer a credencial.

```
secret = SecretClient(  
    vault_url = f"https://{vault}.vault.azure.net",  
    credential = DefaultAzureCredential()  
)  
.get_secret("sql-conn")  
  
conn = pyodbc.connect(secret.value)
```

Cada agente tem sua própria Managed Identity e acessa só o que precisa — **menor privilégio** aplicado a agentes.

LAB 2

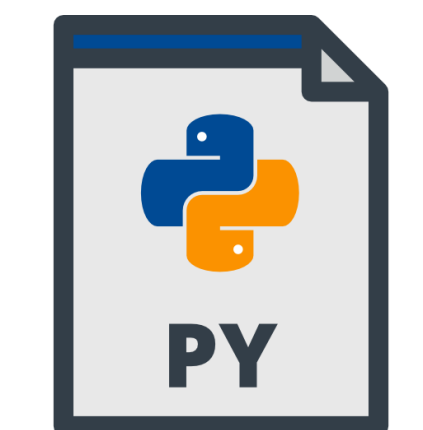


Azure SQL + Key Vault + Python

Antes de continuar, certifique-se de ter à disposição:

- Conta Azure já logada
- Cloud Shell funcionando
- Uma boa xícara de café ☕

Localize o script python popular_produtos.py em lab/scripts usando o VS Code



popular_produtos.py

1. Lê connection string do Key Vault
2. Baixa CSV do Blob Storage
3. Conecta no Azure SQL via pyodbc
4. Cria tabela (drop se existir)
5. Insere produtos

OPCIONAL: para o caso de ter perdido a sessão e variáveis:

```
export KEY_VAULT_NAME=  
export STORAGE_ACCOUNT_NAME=
```

Carga de dados

Instala dependências python

```
pip install --user pyodbc azure-identity azure-keyvault-secrets azure-storage-blob
```

Altera o diretório corrente

```
cd ~/aie-cloud/aulas/02-storage-bancos/lab/scripts
```

Executa script Python de carga de produtos

```
python3 popular_produtos.py
```



A connection string nunca aparece no script — vem do Vault em runtime, autenticada pela identidade do Cloud Shell.

```
kv_name = os.environ["KEY_VAULT_NAME"]
storage_account = os.environ["STORAGE_ACCOUNT_NAME"]

credential = DefaultAzureCredential()

# 1. Ler connection string do Key Vault
print(f"→ Lendo connection string de {kv_name}...")
kv_client = SecretClient(
    vault_url=f"https://{kv_name}.vault.azure.net",
    credential=credential,
)
secret = kv_client.get_secret("sql-connection-string")
conn_str = secret.value
print("✓ Connection string obtida (não vamos imprimi-la)")
```

► LAB 2 – Consulta de dados no SQL Server (**OPCIONAL**) pela interface gráfica do portal

1 Liberação de IP específico no **Servidor SQL**

sql-qc-o1ak73 | Networking

SQL server

Search

Feedback

Public access Private access Connectivity

Public network access

Public Endpoints allow access to this resource through t following network rules still requires proper authorizatic Public network access

Disable Select

Firewall rules

Allow certain public internet IP addresses to access your resource. [Learn more](#)

+ Add your client IPv4 address (177.145.90.45) + Add a firewall rule

Rule name	Start IPv4 address	End IPv4 address
ClientIPAddress_2026-6-7_17-28-49	177.145.90.45	177.145.90.45
CloudShellAccess	40.71.199.151	40.71.199.151

2 Conexão no **Banco SQL** (precisa do usuário sqladminqc e senha que estão no KV)

sqlldb-qc (sql-qc-o1ak73/sqlldb-qc) | Query edit...

SQL database

Search

Feedback Classic experience

Overview Activity log Tags Diagnose and solve problems Query editor (preview) Mirror database in Fabric (preview) Resource visualizer Settings Data management Integrations Power Platform Security Intelligent performance Monitoring Automation

Query Editor

Run queries on your database directly in the browser [Learn more](#)

Connecting to sqlldb-qc on sql-qc-o1ak73

Microsoft Entra authentication SQL authentication

Username *

Enter username

Password *

Enter password

SELECT TOP 100 * FROM T_PRODUTOS

New query Templates Open in Feedback Sign out

SQL query 1

Run Save as view

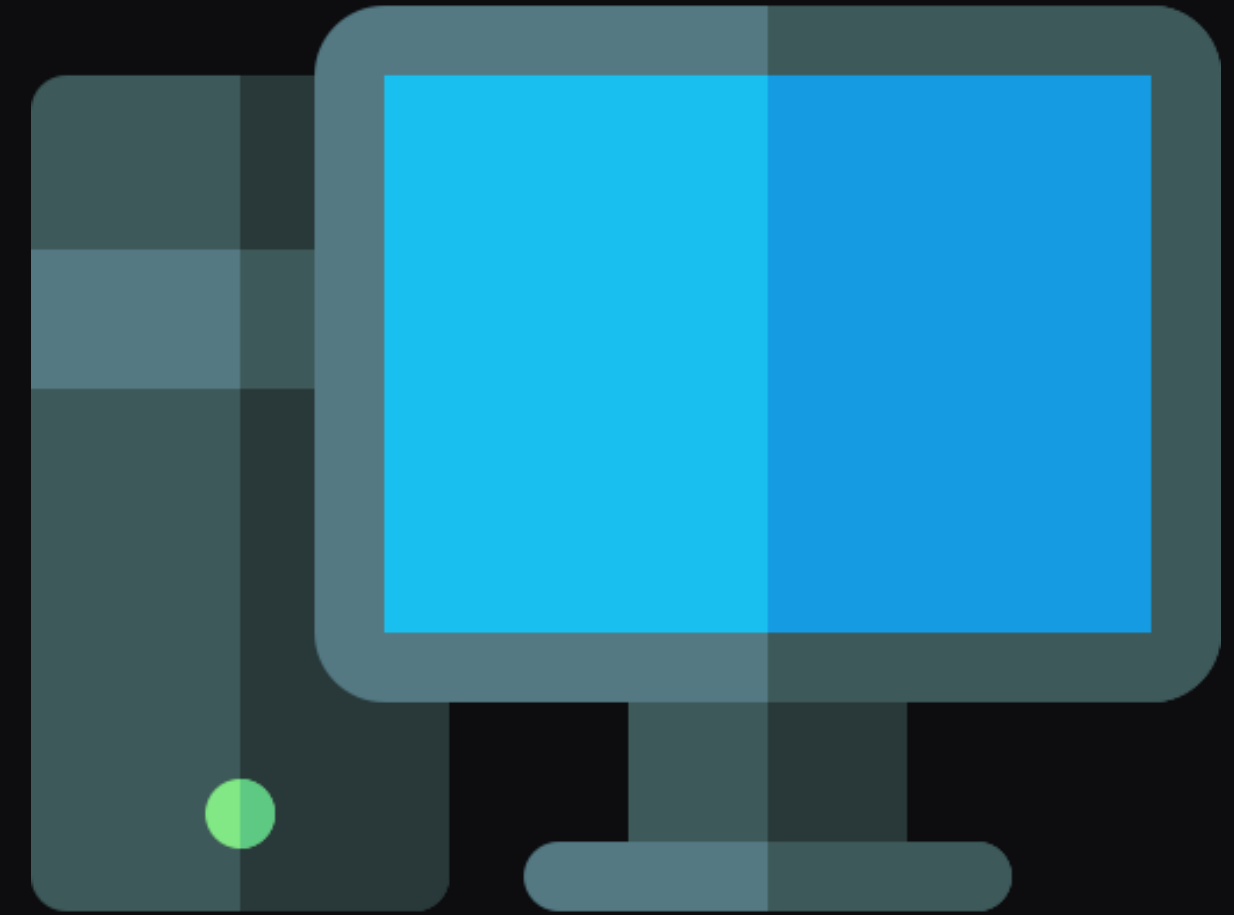
1 SELECT TOP 100 * FROM T_PRODUTOS

	123 id	ABC nome	ABC descricao
1	1	Cadeira Ergonômica DXRacer	Cadeira de escritório com ap...
2	2	Notebook Dell Inspiron 15	Notebook 15.6" Intel Core i5 ...
3	3	Cafeteira Nespresso Essenza ...	Cafeteira de cápsulas compa...
4	4	Tênis Nike Air Zoom Pegasus...	Tênis para corrida com amort...
5	5	Smartphone Samsung Galax...	Smartphone 6.2" 128GB Câma...
6	6	Mesa de Escritório com Regu...	Mesa em pé com motor elétr...

Esse
LAB
me
deixou
ligadão



03



NoSQL & AI Search (busca vetorial)

Famílias de NoSQL, Cosmos DB, e a infraestrutura que torna o RAG dos agentes possível: o Vector Database.

NoSQL não é um banco , é uma classe. As 5 famílias mais populares

DOCUMENTO

JSON Aninhado

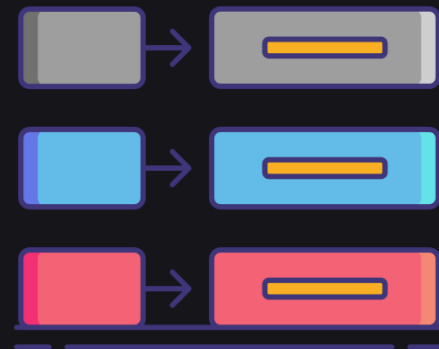


Cosmos DB (SQL API),
MongoDB.

Ideal para domínios
variáveis — reviews,
carrinhos.

KEY-VALUE

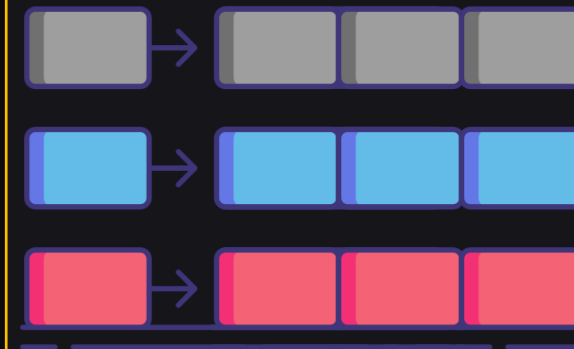
Chave → valor



Redis, DynamoDB.
Caching, sessões, dados
rápidos ou quentes que
expiram.

COLUNAR (Coluna Larga)

Wide-column



Cassandra, DynamoDB,
Cosmos DB (Cassandra API)

Grandes Volumes,
múltiplas colunas,
Telemetria, escala global.

GRAFO

Nós & arestas

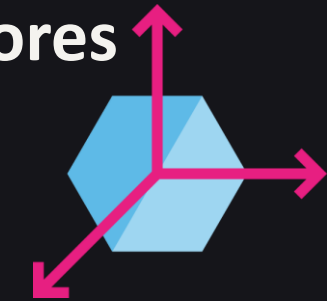


Cosmos Gremlin, Neo4j.

Relações complexas,
fraud detection.

SEMANTIC & VECTOR

Similaridade semântica e vetores



Cosmos NoSQL, Pinecone,
AI Search, Neptune.

DOCUMENTO

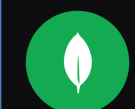
JSON Aninhado



Cosmos DB (SQL API),
MongoDB.

Ideal para domínios
variáveis — reviews,
carrinhos.

Notinha sobre organização:



MongoDB -> Collection

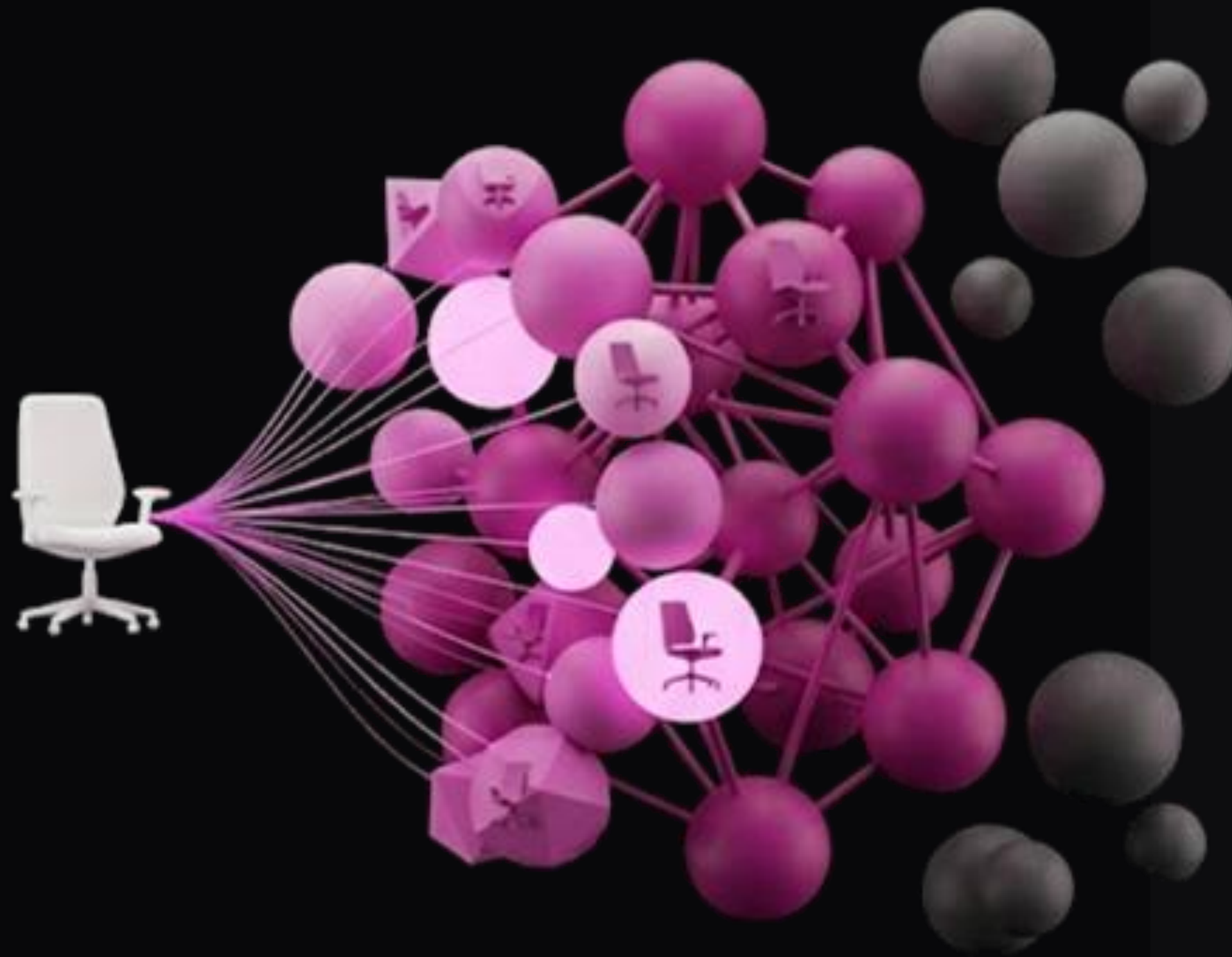


CosmosDB -> Container

```
{
  "id": "r-001",
  "produto_id": "5",
  "score": 5,
  "texto": "Adorei! Chegou rápido e funcionou perfeitamente.",
  "cliente": "cliente-738",
  "_rid": "k1wRAJ2c3dgBAAAAAAAAAA==",
  "_etag": "\"3a00f4d5-0000-0100-0000-665f1c8a0000\"",
  "_attachments": "attachments/",
  "_ts": 1717430922
}
```

```
{
  "id": "r-001",
  "produto": {
    "produto_id": "5",
    "nome": "Fone de Ouvido Bluetooth",
    "categoria": "Eletrônicos",
    "detalhes": {
      "marca": "SoundMax",
      "modelo": "SMX-500",
      "cor": "Preto",
      "especificacoes": {
        "bateria": "20h",
        "conectividade": ["Bluetooth 5.0", "USB-C"],
        "peso": "250g"
      }
    }
  },
  "avaliacao": {
    "score": 5,
    "texto": "Adorei! Chegou rápido e funcionou perfeitamente.",
    "data": "2024-06-03T14:22:00",
    "tags": ["entrega rápida", "qualidade excelente", "recomendado"],
    "detalhes_avaliacao": {
      "embalagem": "Ótima, bem protegida",
      "tempo_entrega_dias": 2,
      "expectativa_vs_realidade": "Superou expectativas"
    }
  },
  "cliente": {
    "id": "cliente-738",
    "nome": "Maria Oliveira",
    "localizacao": {
      "cidade": "São Paulo",
      "estado": "SP",
      "pais": "Brasil"
    }
  },
  "perfil": {
    "idade": 32,
    "compras_anteriores": 12,
    "nivel_fidelidade": "Gold"
  }
}
```

· VIRA A CHAVE ·



“Parecido com isto”
o SQL **não responde.**

Buscar por palavra-chave acha o termo exato.

Buscar por significado é outra categoria de problema
— e é o que faz um agente parecer inteligente.

**“ Encontre os 5 produtos mais parecidos com
cadeira de escritório ergonômica .”**

SQL com LIKE '%cadeira%' não resolve. Busca por keyword não captura semântica.

“Algo para dor nas costas” nem contém a palavra cadeira.

🔍 Busca: "algo para trabalhar em pé"

Resultados:

```
{"nome": "Mesa de Escritório com Regulagem de Altura",  
  "descricao": "Mesa em pé com motor elétrico e tampo 140x70cm"  
}  
{"nome": "Cadeira Ergonômica DXRacer",  
  "descricao": ""  
}
```

Score: 1.42

Score: 0.88

Ranking por significado, não por palavra exata. Modelos de linguagem do provedor (Microsoft) reordenam o resultado do texto que você já indexou.

COMO FUNCIONA

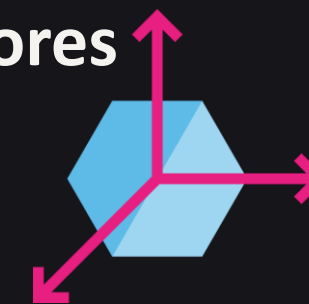
1. A busca tradicional (BM25) acha candidatos por palavras-chaves, similaridade e sinônimos.
2. O semantic ranker relê os candidatos e os reordena por relevância de significado.
3. Devolve @search.reranker_score (0–4) além do @search.score do BM25.

→ Não muda o que está indexado (continua texto). Muda QUEM aparece no topo.

→ SKU do Azure AI Search já inclui semantic ranker.

SEMANTIC & VECTOR

**Similaridade
semântica e
vetores**



Cosmos NoSQL, Pinecone,
AI Search, Neptune.

🔍 Busca: "algo para trabalhar em pé"

0.0123	-0.0456	0.0789	...	0.0021
--------	---------	--------	-----	--------

Documentos existentes:

{ "nome": "Mesa de Escritório com Regulagem de Altura",
 "descricao": "Mesa em pé com motor elétrico e tampo 140x70cm"
 }

-0.0337	0.0654	0.0190	...	-0.0712
---------	--------	--------	-----	---------

{ "nome": "Cadeira Ergonômica DXRacer",
 "descricao": ""
 }

0.0110	-0.0488	0.0801	...	-0.8703,
--------	---------	--------	-----	----------

O texto vira um vetor de N dimensões (embedding). A busca compara vetores por distância — não por palavra.

COMO FUNCIONA

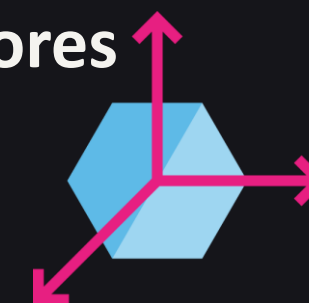
1. Um modelo de embedding (ex.: text-embedding-3-small, 1536 dims) transforma cada texto num vetor. Significados parecidos ficam próximos no espaço.
2. Indexo o documento JÁ com o vetor num campo do tipo Collection(Single).
3. Na busca, a PERGUNTA também vira vetor; o índice devolve os k vizinhos mais próximos por similaridade de cosseno (@search.score).

→ Acha por significado mesmo sem palavra em comum.

→ Requer gerar embeddings (Azure OpenAI) — fechamos esse ciclo na Aula 4.

SEMANTIC & VECTOR

Similaridade semântica e vetores



Cosmos NoSQL, Pinecone, AI Search, Neptune.

Texto vira vetor, busca vira geometria

Embeddings transformam texto e imagem em vetores. Buscar = achar os vetores mais próximos no espaço (ANN — Approximate Nearest Neighbor).

Hybrid search — vector + keyword + semantic ranker. A Microsoft **mediu +30% de relevância nos benchmarks.**

AZURE AI SEARCH

Padrão de mercado em Azure — hybrid: vector + keyword + semantic.

PGVECTOR

Extensão do PostgreSQL — vetor como tipo de coluna.

PINECONE · WEAVIATE · QDRANT

SaaS especializados em nearest-neighbor.

COSMOS DB VECTOR

Vector search nativo — recurso recente (2024), em preview.

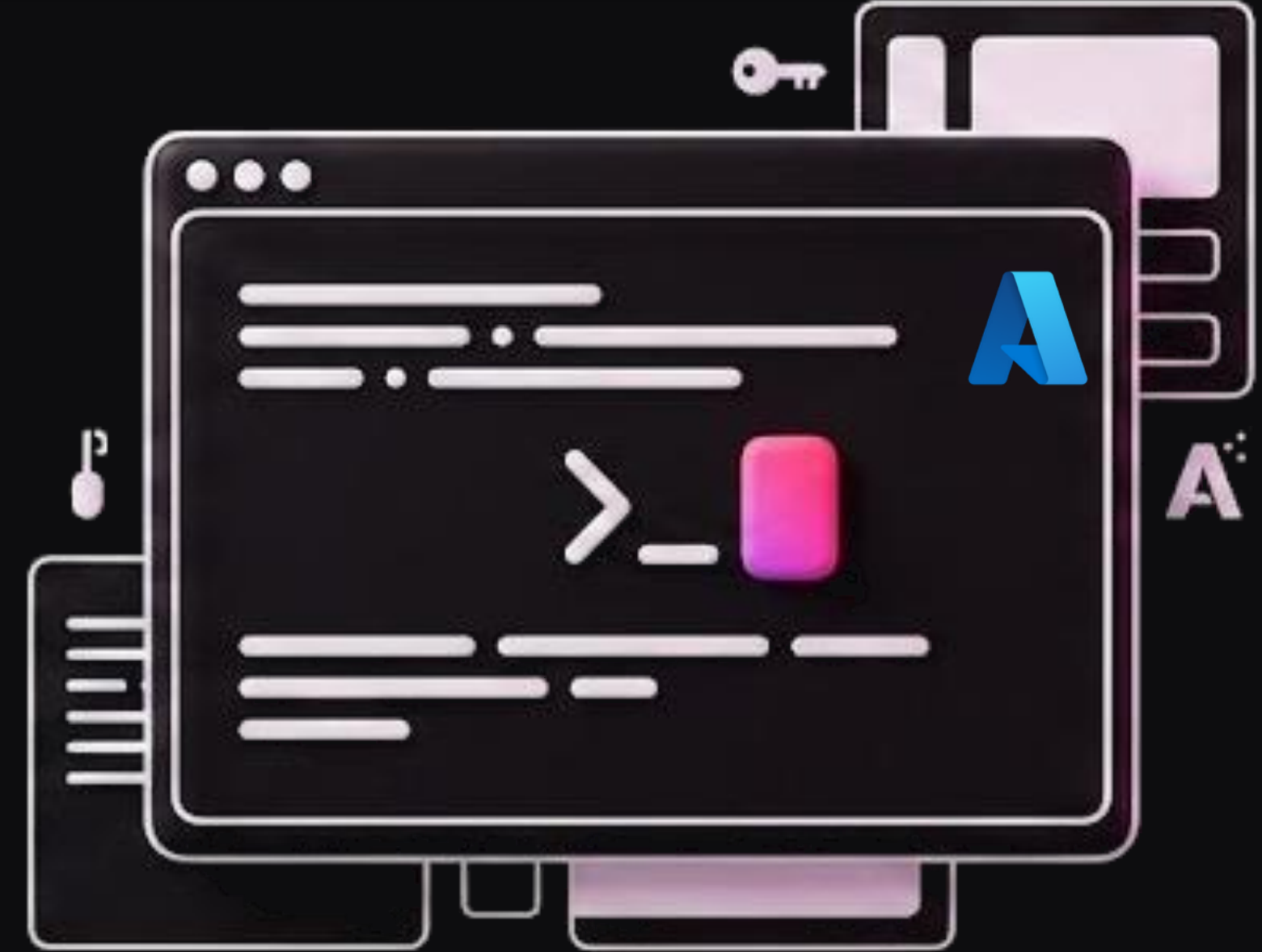
Hoje no lab usamos Semantic. N3 do exercício faz Vector verdadeiro

Como o agente usa tudo isso

01 · PERGUNTA	02 · BUSCA	03 · CONTEXTO	04 · RESPOSTA
Usuário pergunta	Vector DB / Semantic DB	Monta o prompt	LLM responde
“Tem cadeira boa pra coluna?”	recupera os produtos semanticamente próximos	injeta os resultados como contexto relevante	resposta fundamentada nos dados da QC

Esta disciplina provisiona a **infraestrutura** do Vector DB. Embeddings, chunking e prompt são da disciplina de Knowledge Management.

LAB 3



Cosmos DB + Azure AI Search

Antes de continuar, certifique-se de ter à disposição:

- Conta Azure já logada
- Cloud Shell funcionando
- Uma boa xícara de café ☕

Cosmos DB + Azure AI Search

PARTE A · COSMOS · ~20 MIN

Reviews dos clientes

Terraform: conta free tier + container reviews
particionado por /produto_id.

Python

- insere 30 reviews (texto + score 1–5)
- consulta as positivas de um produto.

PARTE B · AI SEARCH · ~25 MIN

Índice de produtos

Terraform: search_service SKU + semantic search.

Python

- cria produtos-index
- indexa os 20 produtos
- busca por significado

⚠ NOTA DE ESCOPO

Hoje é **semantic search**, não vector.
Vector real (embeddings) é o exercício
3.1.

✓ CHECKPOINT L₃

Reviews no Data Explorer e buscas
semânticas no Search Explorer?

Localize o script python popular_reviews.py em lab/scripts usando o VS Code



1. Lê connection string do Key Vault
2. Conecta no Cosmos via CosmosClient
3. Insere 30 reviews aleatórios
 - Na collection/container “reviews”
4. Exibe reviews 4+ do produto 5

OPCIONAL: para o caso de ter perdido a sessão e variáveis:

```
export KEY_VAULT_NAME=  
export COSMOS_ENDPOINT=
```

Carga de dados:

Instala dependências python

```
pip install --user azure-identity azure-cosmos
```

Altera o diretório corrente

```
cd ~/aie-cloud/aulas/02-storage-bancos/lab/scripts
```

Executa script Python de carga de produtos

```
python3 popular_reviews.py
```

► LAB 3A – Consulta de dados no COSMOS (OPCIONAL) pela interface gráfica do portal

COSMOS DB

Home > Resource Manager | Resource groups > rg-qc-aula02-engy0q > cosmos-qc-engy0q

cosmos-qc-engy0q | Data Explorer

Azure Cosmos DB account

Search

Overview
Activity log
Access control (IAM)
Tags
Diagnose and solve problems
Quick start
Data Explorer
Mirroring in Fabric
Container Copy
Resource visualizer
Settings
Integrations
AI Capabilities (Preview)
Containers
Monitoring
Automation
Help

+ New... ▾

Search databases only ▾

Home
qc-db
reviews
Items
Settings
Stored Procedures
User Defined Functions
Triggers

SELECT * FROM c

Type a query predicate (e.g., WHERE c.id="1"), or choose one from the drop down list, or leave empty to query all documents.

	id	/produto_id	score
☐	r-001	4	5
☐	r-002	9	4
☐	r-003	5	5
☐	r-004	18	5
☒	r-005	14	5
☐	r-006	3	4
☐	r-007	17	5
☐	r-008	18	4
☐	r-009	18	1
☐	r-010	15	5
☐	r-011	1	4
☐	r-012	14	2
☐	r-013	5	4
☐	r-014	11	5

```
1 {  
2   "id": "r-005",  
3   "produto_id": "14",  
4   "score": 5,  
5   "texto": "Adorei! Chegou rápido  
6   "cliente": "cliente-130",  
7   "_rid": "ZWkoAP5bsBUFAAAAAAAAAA=  
8   "_self": "dbs/ZWkoAA==/colls/ZWk  
9   "_etag": "\"7200ec2b-0000-0300-0  
10  "_attachments": "attachments/",  
11  "_ts": 1780878588  
12 }
```

Localize o script python `indexar_produtos.py` em `lab/scripts` usando o VS Code



`indexar_produtos.py`

1. Cria índice
2. Baixa CSV do Blob Storage
3. Indexa no AI Search
4. Demonstra buscas

OPCIONAL: para o caso de ter perdido a sessão e variáveis:

```
export SEARCH_ENDPOINT=  
export STORAGE_ACCOUNT_NAME=
```

Carga de dados

Instala dependências python

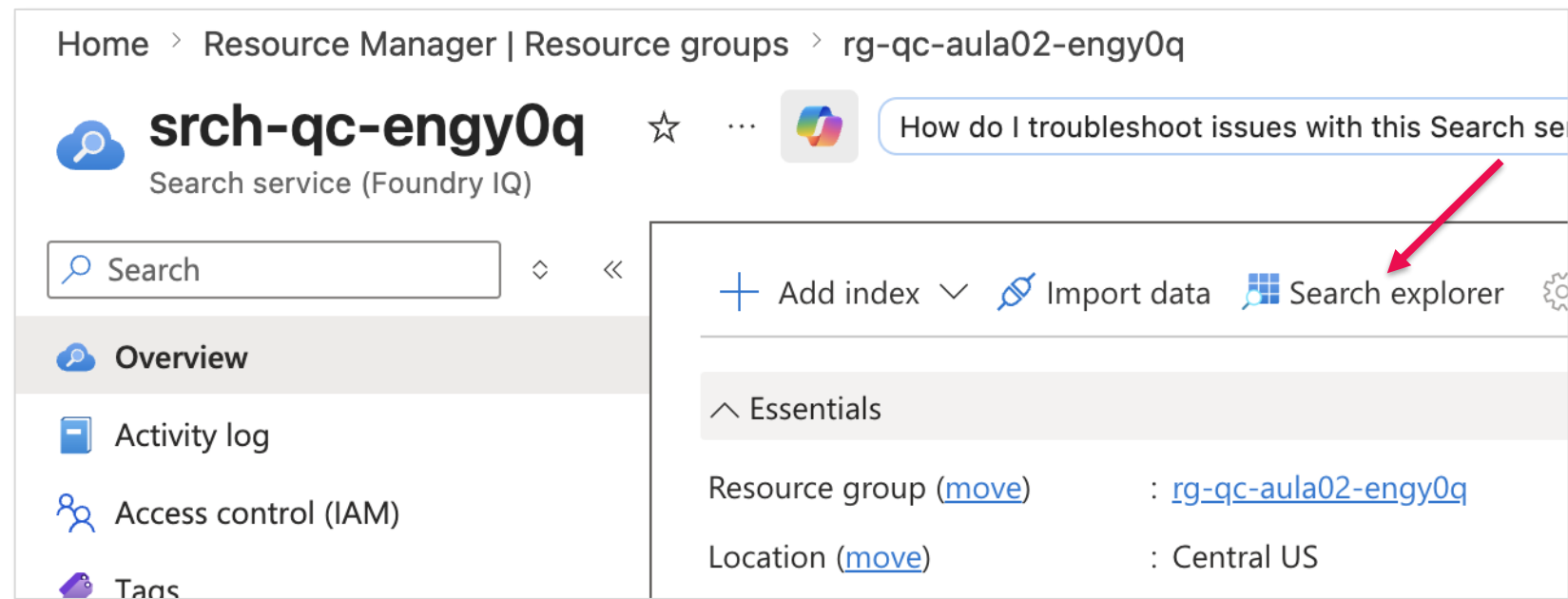
```
pip install --user azure-identity azure-search-documents azure-storage-blob
```

Altera o diretório corrente

```
cd ~/aie-cloud/aulas/02-storage-bancos/lab/scripts
```

Executa script Python de carga de produtos

```
python3 indexar_produtos.py
```



Buscar por

- cadeira escritório
- dor nas costas
- produto para dor nas costas

MONGO DB = ALTERNATIVA
PARA O COSMOS DB

Localize o script python popular_reviews_mongo.py em lab/scripts usando o VS Code



popular_reviews_mongo.py

1. Lê connection string do Key Vault
2. Conecta no MongoDB via MongoClient
3. Insere 30 reviews aleatórios
 - Na collection/container “reviews”
4. Exibe reviews 4+ do produto 5

OPCIONAL: para o caso de ter perdido a sessão e variáveis:

```
export KEY_VAULT_NAME=  
export MONGO_IP=
```

Carga de dados:

Instala dependências python

```
pip install --user pymongo
```

Altera o diretório corrente

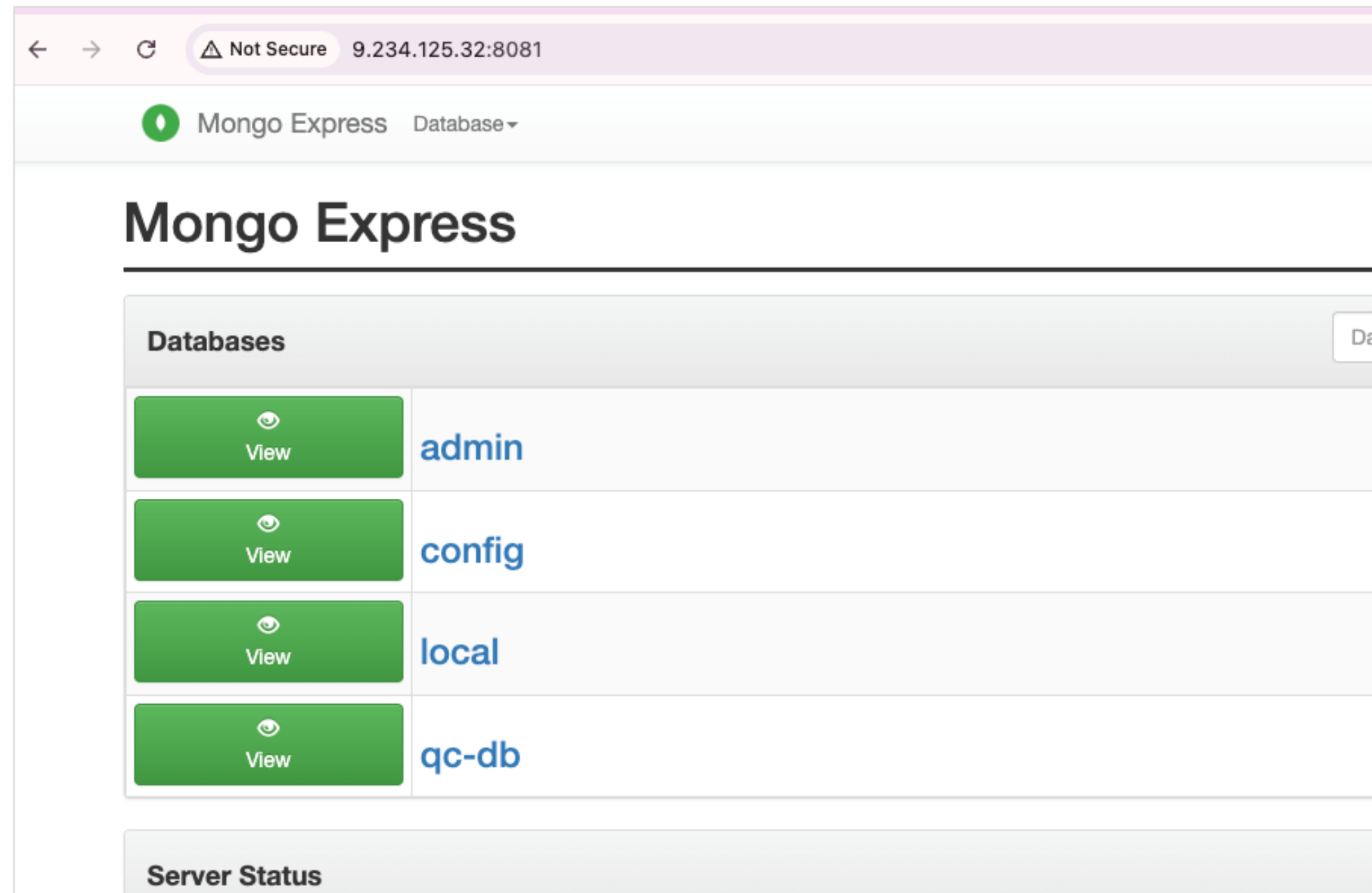
```
cd ~/aie-cloud/aulas/02-storage-bancos/lab/scripts
```

Executa script Python de carga de produtos

```
python3 popular_reviews_mongo.py
```

MONGO DB = ALTERNATIVA
PARA O COSMOS DB

Explore o Mongo Express (UI) pelo IP Público, porta 8081



04



Analytics & encerramento

OLTP vs OLAP, uma menção a Synapse e Fabric, o destroy obrigatório e a arquitetura de dados que ficou pronta hoje.

Online Analytical Processing.

Poucas queries, mas que varrem milhões de linhas pra agregar: somas, médias, tendências

O banco que responde a "quanto vendemos por categoria no último trimestre?"



AZURE SYNAPSE — SERVERLESS SQL POOL

1. Não tem servidor ligado: você consulta arquivos (CSV/Parquet) direto no Data Lake / Blob.
2. Armazenamento COLUNAR — lê só as colunas da query, comprime muito → varredura barata.
3. Cobrança por dado lido na query (pay-per-query), não por hora de máquina.

→ Sintaxe T-SQL conhecida: OPENROWSET aponta pro arquivo no Storage.

→ Ideal pra Big Data analítica

Onde os dados vivem ≠ onde se analisa

OLTP

Transacional — o dia a dia

Azure SQL, Cosmos DB. Operação em tempo real, baixa latência por transação. É o que provisionamos hoje.

Pergunta: "qual o estoque do produto 5?"

Operações: muitas, pequenas (INSERT/UPDATE)

Linhas: 1 a poucas

Dados: atuais

Storage: por LINHA (row)

Latência: milissegundos

Normaliz.: alta (3FN)

● pedido, carrinho, review

OLAP

Analítico — agregações

Dashboards, ML, grandes volumes. **Lakehouse** = Delta/Parquet sobre Object Storage, SQL analítico sem ETL pesado.

Pergunta: "quanto vendi por categoria?"

Operações: poucas, pesadas (SELECT + agreg.)

Linhas: milhões por query

Dados: históricos

Storage: por COLUNA (column)

Latência: segundos a min.

Normaliz.: baixa (star)

● análises e big data

A competência do AI Engineer é saber onde o dado vive e como acessá-lo para alimentar agentes — não construir o DWH. Synapse fica no exercício opcional 3.2 .

· REGRA DE OURO ·

terraform destroy. Toda aula.

O que sobe, desce. Free tiers + lab curto = custo próximo de zero.

O hábito de destruir é o que separa um profissional de quem toma susto na fatura.

—

► BLOCO 4 · DESTROY

terraform destroy.

Toda aula.

1 – Destruir tudo o que foi criado

Destruir TUDO ao final (regra de ouro — custo quase zero)
`terraform destroy`

*Alternativa: Excluir todo o Resource Group
com tudo o que tem dentro*



EXERCÍCIO

Exercício Aula 2

Exercício que vale 10% da nota

Esta é a segunda entrega de grupo da disciplina. Ao todo são 5 entregas intermediárias (10% cada) + projeto integrado final entregue como ZIP 1 semana após a Aula 6 (50%, sem apresentação oral).

Como o grupo se organiza

Os 3 níveis de exercícios são divisão de trabalho dentro do grupo, não escolha individual livre:

 *Nível 1 — Básico: consolidação dos conceitos da aula (storage, tiers, relacional vs NoSQL, RBAC do Key Vault)*

 *Nível 2 — Intermediário: bloco do projeto Quantum Commerce Exercícios 2.1 (matriz de dados QC + diagrama), 2.2 (plano de migração), 2.3 (particionamento Cosmos ou Mongo)*

 *Nível 3 — Avançado: bônus opcional — Exercícios 3.1 (vector search verdadeira), 3.2 (Synapse serverless), 3.3 (benchmark)*

Mínimo obrigatório: N1 + N2 cobertos. N3 é bônus (até +2 pts extras na nota da aula).

Repo Base: <https://github.com/elthonf/aie-cloud.git>

Item	Obrigatório?	Pontos máximos
Cabeçalho do grupo + distribuição do trabalho	✔ Sim	1 pt (Critério 4)
🟢 N1 — Exercícios 1.1, 1.2, 1.3, 1.4 respondidos	✔ Sim	3 pts (Critério 1)
🟡 N2 — Exercícios 2.1 (matriz de dados e diagrama), 2.2 (plano de migração), 2.3 (particionamento Cosmos ou Mongo)	✔ Sim	3 pts (Critério 2) + 2 pts qualidade técnica (Critério 3)
🔴 N3 — Exercícios 3.1 (vector search verdadeira), 3.2 (Synapse serverless), 3.3 (benchmark)	🎁 Bônus	até +2 pts extras
Reflexão coletiva ao final	✔ Sim	1 pt (Critério 5)
Total		10 pts

Sugestão de nome de arquivo: `entrega-grupo-NN-aula01.zip` (substitua NN pelo nome de um integrante).

O que deve conter no arquivo

```
qc-grupo-NN-aula02/
├─ entrega-grupo-aula02.md    # ★ documento principal (template em ../template-entrega-grupo.md)
├─ diagramas/
|   └─ arquitetura-qc-aula02.png # Diagrama do Exercício 2.1
├─ scripts/                   # Python: vector search; Apenas se entregou N3 (Exercícios 3.X)
├─ terraform/                 # Apenas se entregou N3 (Exercícios 3.X)
|   └─ main.tf
|   └─ (outros).tf
└─ README.md                  # Como rodar o N3 (se incluído)
```

Continua...

Pra próxima

Azure Functions consumindo o T_PRODUTOS — a primeira API da QC.
O endpoint que os agentes vão chamar.

O que vocês tiram daqui

BLOB STORAGE

catalogo · imagens · logs

produtos.csv → AI Search

logs → Synapse (futuro)

COSMOS DB

reviews

base da análise de sentimento (Aula 4)



AZURE SQL

T_PRODUTOS

consumido pelos agentes via Function (Aula 3)

AI SEARCH

produtos-index

o RAG dos agentes nasce aqui



KEY VAULT

segredos de tudo

connection strings e chaves nunca tocam o código. Toda a arquitetura acima depende dele.



Obrigado.

Nos vemos na Aula 02 — Armazenamento & Bancos de Dados.